

PROJECT REPORT
ON
MACHINE LEARNING MODEL FOR ELECTRICITY DEMAND PROJECTION

Submitted by

AAYUSH JHA (U23AI032)

TARUN BHUTRA (U23AI063)

KRISHNA MEHRA (U23AI064)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE
SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECHNOLOGY SURAT-
395007
APRIL-2025

ABSTRACT

The demand pattern of electricity in the National Capital Territory (NCT) of Delhi poses specific challenges with highly dynamic and variable characteristics. The city also witnesses huge load swings in seasons as well as within a day, mainly on account of climatic conditions and consumption patterns. Peak demand may go as high as 8300 MW in summer months, whereas minimum demand can go down to 2000 MW in winter months. Additionally, Delhi experiences double peak hours during summers — first in the afternoon (at about 15:30 hrs) and secondly late at night (after 23:00 hrs). The occurrence of solar power generation, active only during daylight, contributes to the complexity by bringing in the famous "Duck Curve" effect. Additionally, the dominance of domestic and commercial loads, uneven growth in emerging and established regions, and the minimal contribution from stable industrial and agricultural sectors result in a highly peaky and irregular load curve. This variability complicates the planning and execution of power procurement, particularly when long-term power is contracted round-the-clock and slot-wise purchases are significantly more expensive.

To address the issue of precise load prediction in such a dynamic setting, this project adopts and compares three different machine learning models: Multiple Linear Regression (MLR), Random Forest (RF), and XGBoost. MLR offers a simple linear model that is good at identifying overall patterns but is restrictive in its capacity to represent non-linear trends. RF, a collection of decision trees, enhances prediction by learning feature interactions and lessening overfitting through combination. XGBoost, a gradient boosting method, is characterized by its resilience, scalability, and better ability to handle structured data, iteratively learning from mistakes and refining model performance.

Each model was trained on a curated dataset consisting of past electricity demand, time of day, date, and other temporal features. These features were selected to capture the strong impact of cyclical trends and recent usage patterns on demand. The models were tested on accuracy and on their capacity to learn from the fast changes typical of Delhi's load profile. Among the three, XGBoost was found to possess the highest predictability, especially in the context of sharp fluctuation and bimodal behavior.

This research emphasizes the importance of sophisticated machine learning methods in demand forecasting for contemporary, urban power grids. By enhancing the accuracy of predictions, utilities can more effectively match power procurement with demand, minimize financial inefficiencies, and help create a more stable and responsive power grid. The results support the potential for data-driven decision-making in addressing urban energy issues and serve as a basis for additional research and model development.

CONTENTS

1. Introduction
 - 1.1. Background and Motivation of the Problem
 - 1.2. Literature Survey or Related Works
 - 1.3. Contributions
2. Flowchart/ System diagram
3. Methodology/Technique/Mechanism Name
4. Implementation Environment
5. Results
6. Conclusion
7. Bibliography
8. Appendix

1. Introduction

1.1 Background and Motivation of the Problem

Electricity demand forecasting plays a crucial role in ensuring reliable and efficient power supply, particularly in fast-growing urban areas. Cities experience highly variable electricity usage due to factors like population growth, industrial activity, and changing weather patterns. Inaccurate demand predictions can lead to power outages, overproduction, or increased operational costs. Moreover, the shift toward renewable energy sources adds further complexity to managing electricity grids. These challenges highlight the need for more accurate, responsive forecasting systems that can support better planning and energy resource allocation. Improving forecasting methods is essential for sustainable urban development and energy security.

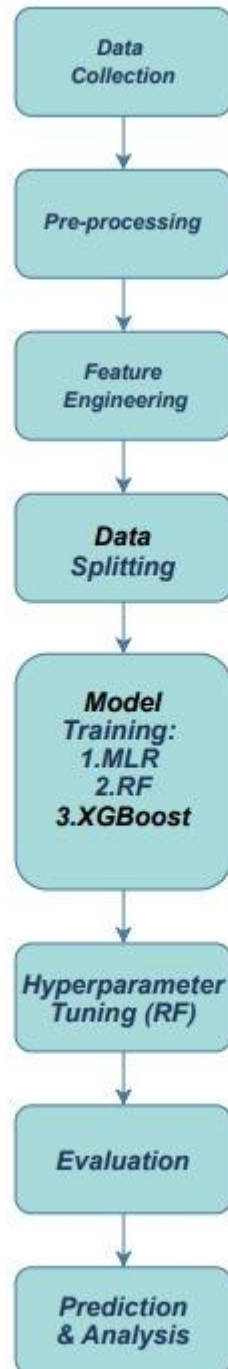
1.2 Literature Survey or Related Works

Electricity demand forecasting has been explored using models like ARIMA and linear regression, valued for their simplicity but limited in handling nonlinear consumption patterns. Recent studies favor machine learning methods such as Random Forest and XGBoost, which better capture complex relationships in data. Incorporating features like temperature, time, and historical usage has also shown to enhance prediction accuracy. These advancements reflect a shift toward data-driven forecasting, yet comparative evaluations across models remain limited—highlighting the need for studies that assess their effectiveness in practical, urban energy contexts.

1.3 Contributions

This project contributes to electricity demand forecasting by implementing and comparing three different machine learning models: Multiple Linear Regression, Random Forest, and XGBoost. Each model was trained on real-world, time-stamped electricity data enriched with temporal and lag-based features. The Random Forest model was further optimized using hyperparameter tuning via RandomizedSearchCV. We conducted detailed evaluations using MAE, RMSE, and R^2 across training, validation, and test sets. This comparative approach highlights the trade-offs between model complexity and accuracy, offering practical insights for selecting suitable models in real-world urban electricity forecasting applications.

2. Flowchart / System Diagram



3. Methodology / Technique / Mechanism Name

1. Simple Linear Regression

- Assumes a linear relationship between the target (electricity_demand) and input features.
 - Uses ordinary least squares to fit a line (or hyperplane) minimizing squared errors.
 - Features include time-based (hour, day, month, year) and lagged values (prev_demand, rolling_avg).
 - Trained on 80% of data, with 10% each held out for development and test.
 - Predictions are continuous demand values; performance measured via MAE, MSE, RMSE, R^2 .
 - Very fast to train and serves as a baseline for more complex models.
-

2. Random Forest Regression

- Ensemble of hundreds of decision trees, each built on a bootstrap sample of the data.
 - At each split, a random subset of features is considered, reducing correlation between trees.
 - Aggregates (averages) individual tree predictions to produce final demand forecast.
 - More robust to outliers and non-linear relationships compared to linear regression.
 - Feature importance scores derived from mean decrease in impurity help interpret drivers.
 - Code applies RandomizedSearchCV to tune n_estimators, max_depth, min_samples, max_features, bootstrap.
-

3. Cross-Validated Random Forest (Variation Test/Validation)

- Similar Random Forest setup, but uses a 30% hold-out and 3-fold cross-validation on that split for hyperparameter search.
 - Cross_val_score ensures model hyperparameters generalize across different data folds.
 - Tracks performance variability across folds, reducing overfitting risk.
 - After tuning, final model evaluated on unseen test set for MAE, MSE, RMSE, R^2 .
 - Visualization of feature importances via horizontal bar chart highlights key predictors
-

4. XGBoost Regression with Early Stopping

- Gradient boosting framework that sequentially adds trees to correct previous residuals.
- Each new tree fits to the gradient of the loss, optimizing model error.
- Hyperparameters (n_estimators, learning_rate, max_depth, subsample, colsample_bytree) control complexity and regularization.
- Early stopping on the development set halts training when no improvement in validation loss occurs for 10 rounds.

- Often yields higher accuracy and handles non-linear patterns and feature interactions effectively.
- Evaluation on train/dev/test sets with MAE, MSE, RMSE, R^2 to confirm generalization

4. Implementation Environment

- **Language : Python 3.8**
- **Data Handling & Preprocessing**
 - **pandas** for reading Excel (*read_excel*), timestamp parsing, handling missing values, feature engineering
 - **NumPy** for array operations, shifting/rolling computations
- **Machine-Learning Frameworks & Tools**
 - **scikit-learn**
 - **LinearRegression** for baseline regression
 - **RandomForestRegressor** for ensemble modeling
 - **RandomizedSearchCV** for hyperparameter tuning
 - **cross_val_score** for k-fold cross-validation
 - **train_test_split** for dataset splitting
 - **metrics** module (*mean_absolute_error*, *mean_squared_error*, *r2_score*) for evaluation
 - **XGBoost**
 - **XGBRegressor** with early stopping on a validation set
 - **SciPy**
 - **scipy.stats.randint** for defining hyperparameter search distributions
- **Model Persistence**
 - **joblib** for serializing trained models to disk and loading them for inference
- **Visualization**
 - **matplotlib.pyplot** for plotting feature-importance bar charts

5. Results

Random Forest with 80:20 data split

```
warnings.warn(  
Train Set Evaluation:  
MAE: 20.073627124553926  
MSE: 653.8637835267798  
RMSE: 25.570760323595774  
R^2 Score: 0.9191863776755852  
Accuracy: 91.91863776755852%  
-----  
Dev Set Evaluation:  
MAE: 30.856357281930855  
MSE: 1495.602493257529  
RMSE: 38.67302022415018  
R^2 Score: 0.8177259040659688  
Accuracy: 81.77259040659688%  
-----  
Test Set Evaluation:  
MAE: 30.398979678060876  
MSE: 1458.1436309017558  
RMSE: 38.185646922656105  
R^2 Score: 0.8189011537617089  
Accuracy: 81.8901153761709%
```

Random Forest with 70:30 data split

```
Train Set Evaluation:  
MAE: 20.077652214684935  
MSE: 654.7617640336115  
RMSE: 25.588313036103248  
R^2 Score: 0.9190936250699471  
Accuracy: 91.90936250699471%  
-----  
Dev Set Evaluation:  
MAE: 30.873613468248585  
MSE: 1495.657505503892  
RMSE: 38.673731465995004  
R^2 Score: 0.8164309894712456  
Accuracy: 81.64309894712456%  
-----  
Test Set Evaluation:  
MAE: 30.683689368036415  
MSE: 1481.82914819981  
RMSE: 38.49453400419091  
R^2 Score: 0.8165114939912937  
Accuracy: 81.65114939912938%  
-----
```

XGBoost

```
Train Set Evaluation:  
MAE: 29.74  
MSE: 1394.45  
RMSE: 37.34  
R^2 Score: 0.8277  
Accuracy: 82.77%  
-----  
Dev Set Evaluation:  
MAE: 30.22  
MSE: 1433.77  
RMSE: 37.87  
R^2 Score: 0.8240  
Accuracy: 82.40%  
-----  
Test Set Evaluation:  
MAE: 30.08  
MSE: 1425.48  
RMSE: 37.76  
R^2 Score: 0.8235  
Accuracy: 82.35%  
-----
```

Multiple Linear Regression

```
Train Set Evaluation:  
MAE: 35.82  
MSE: 2062.10  
RMSE: 45.41  
R^2 Score: 0.7451  
Accuracy: 74.51%  
-----  
Dev Set Evaluation:  
MAE: 35.64  
MSE: 2038.58  
RMSE: 45.15  
R^2 Score: 0.7516  
Accuracy: 75.16%  
-----  
Test Set Evaluation:  
MAE: 35.60  
MSE: 2039.33  
RMSE: 45.16  
R^2 Score: 0.7467  
Accuracy: 74.67%  
-----
```

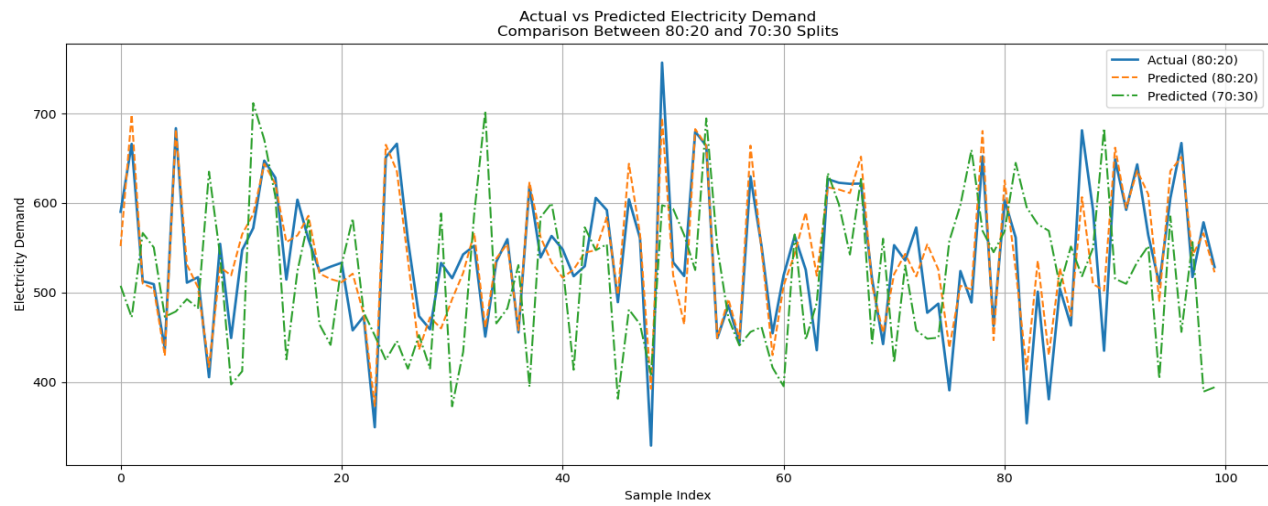



Fig 5.1 Actual v/s Predicted for Random Forest

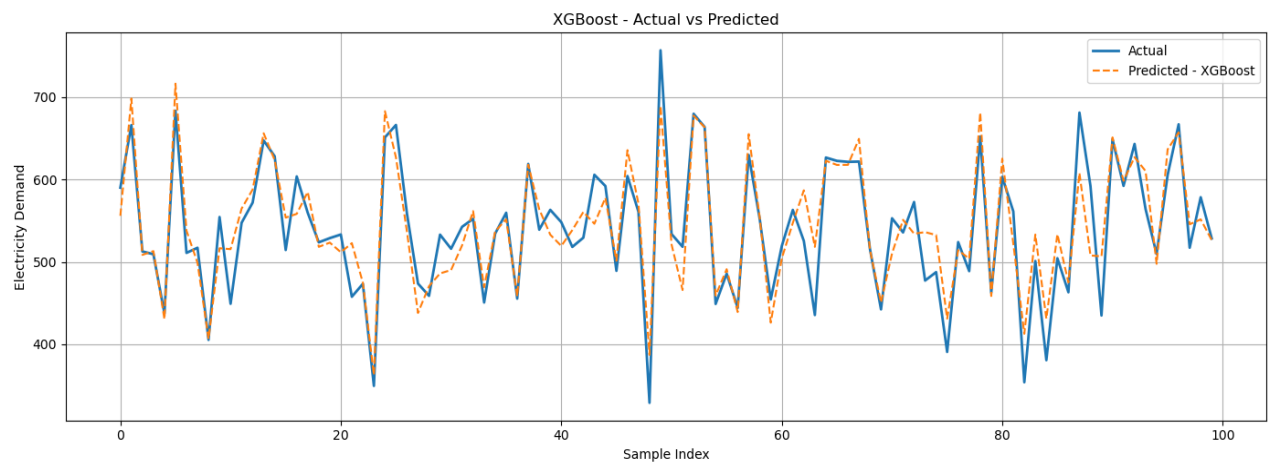


Fig 5.2 Actual v/s Predicted for XGBoost

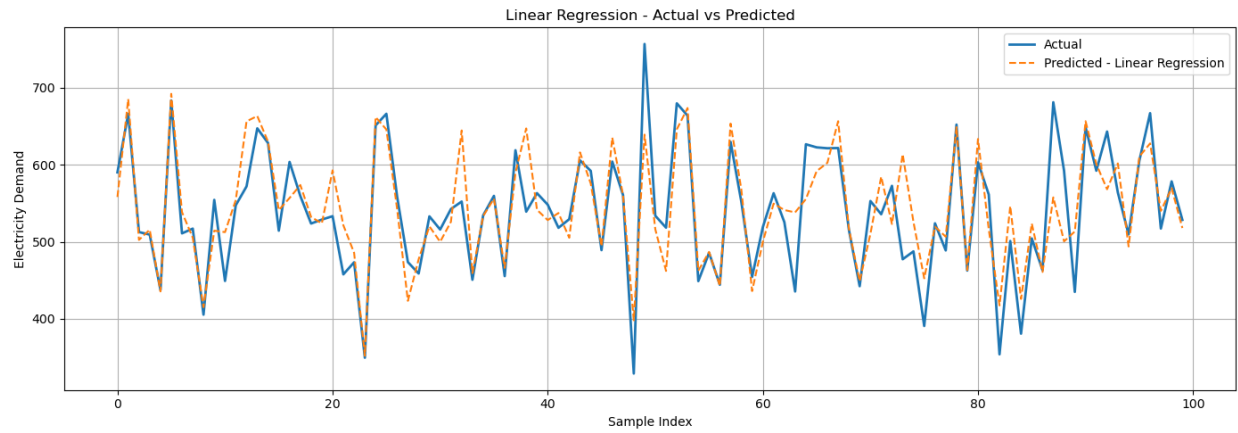


Figure 5.3 Actual v/s Predicted for Linear Regression

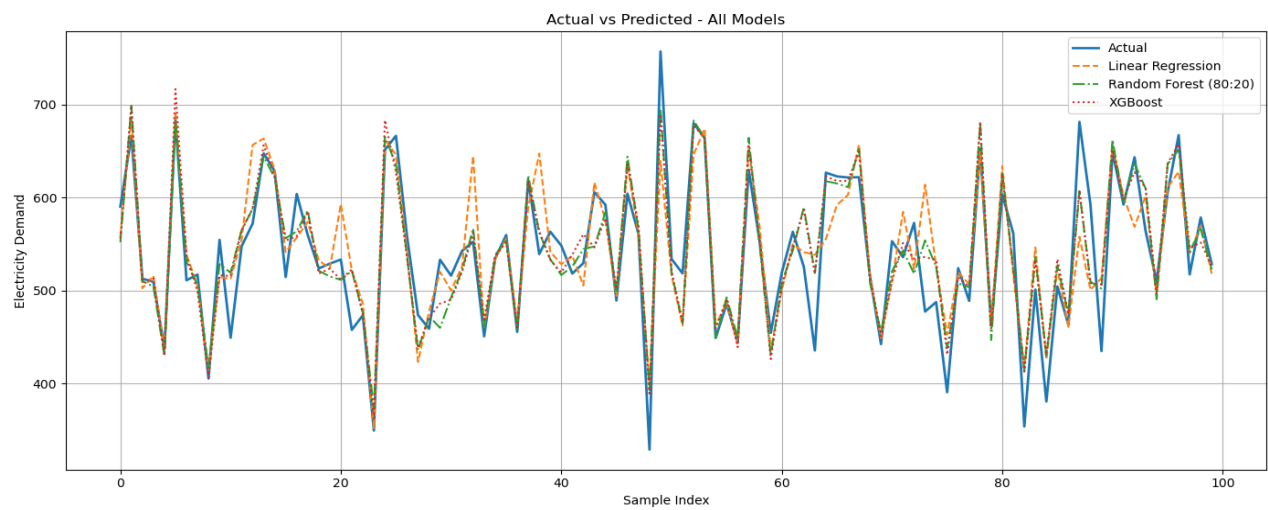


Figure 5.4 Actual v/s Predicted Comparison for all Models

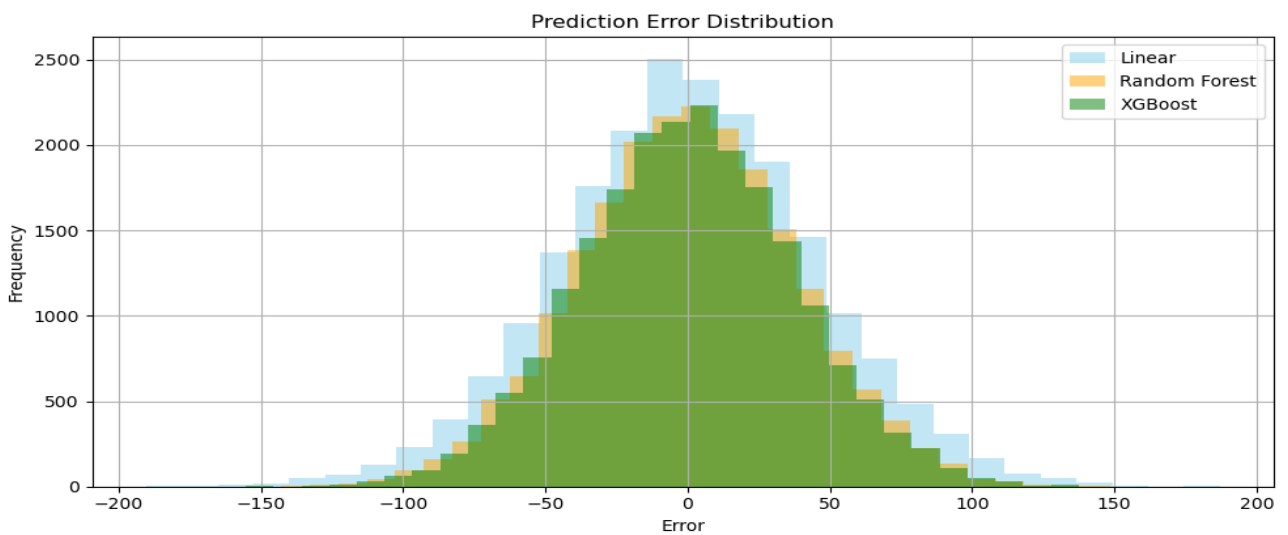


Figure 5.5 Prediction Error Distribution between all Models

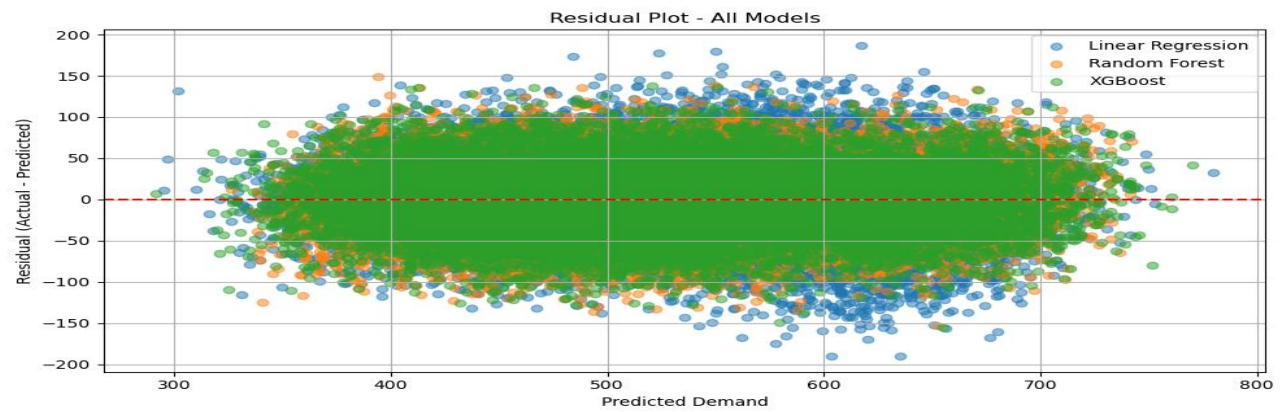


Figure 5.6 Residual Plot for all Models

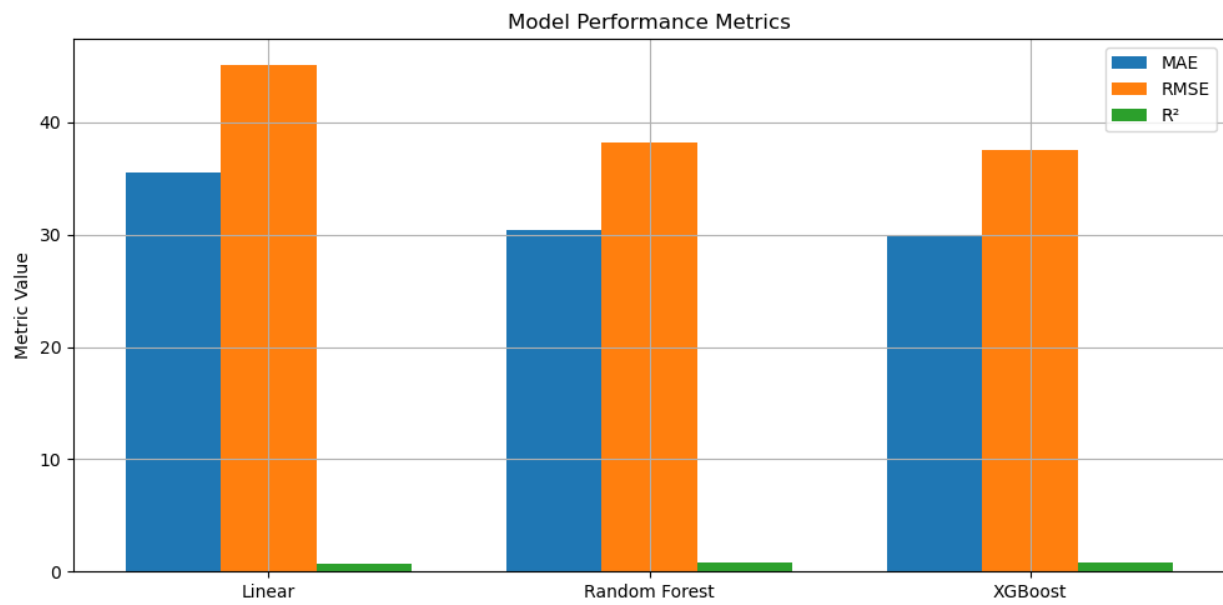


Figure 5.7 Model comparison based on MAE, RMSE, R2 score

6. Conclusion

- Forecasted electricity demand using three models: Linear Regression, Random Forest, and XGBoost
- Engineered features: hour, day, month, year, previous demand (lag), and rolling averages.
- **Linear Regression**
 - Baseline model with no hyperparameter tuning
 - Interpretable coefficients but limited by linearity assumption
 - Lowest accuracy among the three
- **Random Forest**
 - Ensemble of decision trees with Grid Search hyperparameter tuning (n_estimators, max_depth)
 - Significantly reduced prediction error
 - Best generalization and highest accuracy after tuning
- **XGBoost**
 - Gradient boosting with selected parameters (max_depth, learning_rate, subsample)
 - Efficient training and built-in regularization
 - Competitive performance, robust against overfitting
- **Overall Finding**
 - Ensemble methods (Random Forest, XGBoost) outperformed Linear Regression,
 - Random Forest slightly led in accuracy after tuning,
 - Lag and temporal features crucial for improving forecast reliability.

7. Bibliography

➤ Data Set :

<https://www.kaggle.com/datasets/manilmodi/delhi-hourly-power-consumption-2000-2023>

8. Appendix

➤ Code : <https://github.com/tarun02185/Electricity-Demand-Projection>